

VIDEO_EXERCISES

HelixCode Phase 3 - Video Course Exercises

Hands-on exercises to reinforce learning from each video module.

Module 1: Introduction and Overview

Exercise 1.1: Environment Setup

Objective: Set up your development environment for the course.

Tasks: 1. Clone the HelixCode repository 2. Build the project with `make build` 3. Run tests with `make test` 4. Start the server and verify health endpoint

Verification:

```
curl http://localhost:8080/health
# Expected: {"status":"healthy",...}
```

Exercise 1.2: First Session

Objective: Create your first development session.

Tasks: 1. Start the HelixCode CLI 2. Create a session named “hello-world” in building mode 3. Start the session 4. Complete the session 5. View session status

Expected Output:

```
Session 'hello-world' created
Session started
Session completed
Status: completed
```

Module 2: Session Management

Exercise 2.1: Session Lifecycle

Objective: Practice the full session lifecycle.

Tasks: 1. Create sessions for each mode (planning, building, testing, refactoring, debugging, deployment) 2. Practice pause/resume transitions 3. Complete some, fail others 4. Query sessions by status

Code Template:

```
func exercise21() {
    mgr := session.NewManager()

    // Create one session per mode
    modes := []session.Mode{
        session.ModePlanning,
        session.ModeBuilding,
        session.ModeTesting,
        session.ModeRefactoring,
        session.ModeDebugging,
        session.ModeDeployment,
    }

    for _, mode := range modes {
        sess := mgr.Create(
            fmt.Sprintf("%s-practice", mode),
            mode,
            "exercise-project",
        )
        // TODO: Start, pause, resume, complete
    }

    // TODO: Query by status and verify counts
}
```

Exercise 2.2: Multi-Project Sessions

Objective: Manage sessions across multiple projects.

Tasks: 1. Create 3 projects: “api-server”, “web-client”, “shared-lib” 2. Create 2 sessions per project 3. Query sessions by project 4. Export one session to JSON 5. Import it back

Verification Questions: - How many total sessions? - How many active sessions per project? - Did export/import preserve all data?

Exercise 2.3: Session Statistics

Objective: Analyze session usage patterns.

Tasks: 1. Create 10+ sessions with various modes and statuses 2. Get statistics 3. Answer: Which mode is most used? What's the completion rate? 4. Trim history to keep only 5 sessions 5. Verify trim worked

Module 3: Memory and Context

Exercise 3.1: Conversation Building

Objective: Build a multi-turn conversation.

Tasks: 1. Create a conversation titled "API Design Discussion" 2. Add system message setting context 3. Add 5 user/assistant message pairs 4. Search for specific term 5. Get conversation statistics

Code Template:

```
func exercise31() {
    mgr := memory.NewManager()

    conv := mgr.CreateConversation("API Design Discussion")
    conv.SessionID = "design-session-1"

    // Add system context
    mgr.AddMessage(conv.ID, memory.NewSystemMessage(
        "You are an API design expert specializing in REST and GraphQL.",
    ))

    // TODO: Add 5 user/assistant pairs about API design
    // Topics: authentication, pagination, versioning, error
    //         handling, caching

    // TODO: Search for "authentication"
    // TODO: Get and print statistics
}
```

Exercise 3.2: Token Management

Objective: Practice token limits and truncation.

Tasks: 1. Create conversation with 100 message limit 2. Add 150 messages 3. Verify automatic truncation 4. Check token count stays under limit 5. Manually truncate to 20 messages

Exercise 3.3: Conversation Export

Objective: Export and import conversations.

Tasks: 1. Create a meaningful conversation (10+ messages) 2. Export to JSON file 3. Clear the manager 4. Import from JSON 5. Verify all data preserved

Module 4: State Persistence

Exercise 4.1: Save and Load

Objective: Practice basic persistence operations.

Tasks: 1. Create sessions, conversations, and templates 2. Save state to each format (JSON, Compact, GZIP) 3. Compare file sizes 4. Load from each format 5. Verify data integrity

Verification:

```
func verifyData(before, after *State) bool {  
    return before.SessionCount == after.SessionCount &&  
           before.ConversationCount == after.ConversationCount &&  
           before.TemplateCount == after.TemplateCount  
}
```

Exercise 4.2: Auto-Save Configuration

Objective: Configure and test auto-save.

Tasks: 1. Enable auto-save with 30-second interval 2. Make changes every 10 seconds 3. Verify auto-save triggered 4. Disable auto-save 5. Simulate crash and recovery

Exercise 4.3: Backup and Restore

Objective: Implement backup strategy.

Tasks: 1. Create meaningful state (sessions, conversations, templates) 2. Create 3 timestamped backups 3. Make destructive changes 4. Restore from backup 5. Verify restoration

Module 5: Template System

Exercise 5.1: Custom Templates

Objective: Create custom code templates.

Tasks: 1. Create template for HTTP handler 2. Create template for database model 3. Create template for test file 4. Render each with sample variables 5. Verify output correctness

Code Template:

```
func exercise51() {
    mgr := template.NewManager()

    // HTTP Handler template
    handler := template.NewTemplate(
        "HTTP Handler",
        "RESTful HTTP handler",
        template.TypeCode,
    )
    handler.SetContent(`
package handlers

import (
    "net/http"
    "encoding/json"
)

// {{HandlerName}}Handler handles {{Description}}
func {{HandlerName}}Handler(w http.ResponseWriter, r
    *http.Request) {
    switch r.Method {
    case http.MethodGet:
        // TODO: Implement GET
    case http.MethodPost:
        // TODO: Implement POST
    default:
        http.Error(w, "Method not allowed",
            http.StatusMethodNotAllowed)
    }
}
`)

    // TODO: Add variables
```

```
// TODO: Create database model template
// TODO: Create test file template
}
```

Exercise 5.2: Template Library

Objective: Build a reusable template library.

Tasks: 1. Create 5+ templates for common Go patterns 2. Tag appropriately (http, database, testing, etc.) 3. Save to files 4. Load from directory 5. Search and filter

Exercise 5.3: Template Workflows

Objective: Use templates in development workflow.

Tasks: 1. Create session for new feature 2. Use templates to generate: - Handler code - Model code - Test file 3. Store generated code in conversation 4. Export session with conversation

Module 6: Integration

Exercise 6.1: Full Development Workflow

Objective: Implement complete feature using all Phase 3 features.

Scenario: Implement a user registration endpoint

Tasks: 1. Create planning session 2. Plan feature in conversation 3. Switch to building session 4. Use templates for code generation 5. Switch to testing session 6. Document test results 7. Complete workflow

Deliverables: - Session history showing transitions - Conversation with planning and implementation - Generated code from templates - Test results documented

Exercise 6.2: Code Review Workflow

Objective: Implement AI-assisted code review.

Tasks: 1. Create code review session 2. Add code to conversation 3. Use code review template 4. Document findings 5. Export review results

Exercise 6.3: Multi-Session Project

Objective: Manage complex project with multiple sessions.

Scenario: Build a REST API with 3 endpoints

Tasks: 1. Create project structure 2. Create session per endpoint 3. Link conversations to sessions 4. Track progress with tags 5. Generate final report

Final Project

Capstone: AI Development Assistant

Objective: Build a complete development workflow tool using all Phase 3 features.

Requirements: 1. Session management for different work types 2. Conversation tracking for all AI interactions 3. Template library for code generation 4. State persistence with auto-save 5. Export/import capabilities 6. Statistics and reporting

Deliverables: 1. Working code implementation 2. Documentation 3. Test coverage 4. Demo video/screenshots

Grading Criteria: - Functionality (40%) - Code quality (20%) - Documentation (20%) - Test coverage (20%)

Self-Assessment Quiz

Questions

1. What are the 6 session modes in HelixCode?
2. How do you transition a session from paused to active?
3. What's the difference between `Truncate()` and `SetMaxMessages()`?
4. Which persistence format is smallest?
5. How do you add a variable to a template?
6. What callback fires when a session completes?
7. How do you search across all conversations?
8. What happens when token limit is exceeded?
9. How do you export a session?
10. What's the recommended auto-save interval?

Answers

1. Planning, Building, Testing, Refactoring, Debugging, Deployment
2. `mgr.Resume(sessionID)`
3. Truncate immediately removes; SetMaxMessages sets limit for future
4. GZIP (FormatJSONGZIP)

5. `tpl.AddVariable(template.Variable{Name: "name", Required: true})`
 6. `OnComplete(func(s *session.Session))`
 7. `mgr.Search("query")` or `mgr.SearchMessages("query")`
 8. Oldest messages automatically removed
 9. `mgr.Export(sessionID)` returns `SessionSnapshot`
 10. 5 minutes (300 seconds) for most use cases
-

Resources

- [HelixCode Documentation](#)
- [API Reference](#)
- [GitHub Repository](#)
- [Community Forum](#)
- [Video Course](#)